

ASK THE US ECONOMY

A copy-and-paste Snowflake notebook for building a conversational economic-data agent.

WHAT YOU BUILD

Snowflake Public Data → transform economic indicators → Dynamic Table → Semantic View → Cortex Analyst → Streamlit UI.

HOW TO USE THIS GUIDE

Every code cell is selectable and ready to copy. Each section identifies the exact Snowflake or local environment where it runs. Replace placeholder credentials before running local Python.

Contents

- Part 1 Explore the Public Data
- Part 2 Snowpark Connect: Prototype the Transform
- Part 3 Dynamic Tables: Production Pipeline
- Part 4 Semantic Views: Teach AI Your Data
- Part 5 Cortex Analyst and Streamlit
- Completion Architecture and next steps

What You're Building

A conversational AI agent that answers plain-English questions about the US economy • powered entirely by Snowflake.

This notebook walks you through each step. Each section has: • Markdown: Context and instructions • Code: SQL or Python to run for that step (run SQL in Snowsight; Python in your local environment or Snowflake)

Part 1: Explore the Data

We're using Snowflake's free Public Data from the Marketplace • 90+ public domain sources, zero ETL.

Prereqs: Create a free Snowflake account and get the free dataset: [Snowflake Public Data (Marketplace)](<https://app.snowflake.com/marketplace/listing/GZTSZ290BV255/snowflake-public-data-products-snowflake-public-data-free?sortBy=popular&pricing=free&available=available>)

Step 1.1 - Show the database

Run this in a Snowsight worksheet to see that `SNOWFLAKE_PUBLIC_DATA_FREE` is available. *(SQL cells in this notebook are meant to be run in Snowsight; Python cells in your local env or Snowflake.)*

RUN IN: Snowsight SQL Worksheet

```
SHOW DATABASES;
```

Step 1.2 - Preview the economic indicators table

Each row is one data point: a geography, a variable name, a date, and a value. Classic time series format.

RUN IN: Snowsight SQL Worksheet

```
SELECT *  
FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES  
LIMIT 10;
```

Step 1.3 - Look at our three indicators

We'll use: CPI (inflation), Unemployment rate, and 30-year mortgage rate (from a different table).

CPI (inflation) • Consumer Price Index. A value of 324 means prices are about 3.24× the 1982-84 level.

RUN IN: Snowsight SQL Worksheet

```
-- CPI (inflation)  
SELECT VARIABLE_NAME, DATE, VALUE  
FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES  
WHERE GEO_ID = 'country/USA'  
AND VARIABLE_NAME = 'CPI: All items, Monthly, 1982-84 Index Date (Seasonally adjusted)'  
ORDER BY DATE DESC  
LIMIT 12;
```

Unemployment rate • stored as a decimal (e.g.

0.041 =

4.1%).

RUN IN: Snowsight SQL Worksheet

```
-- Unemployment rate
SELECT VARIABLE_NAME, DATE, VALUE
FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES
WHERE GEO_ID = 'country/USA'
AND VARIABLE_NAME = 'Current Labor Force: Unemployment Rate - 20 yrs. & over, Monthly (Seasonally adjusted)'
ORDER BY DATE DESC
LIMIT 12;
```

30-year mortgage rate • from the Freddie Mac housing table (weekly).

0.0626 =

6.26%.

RUN IN: Snowsight SQL Worksheet

```
-- 30-year mortgage rate (different table)
SELECT VARIABLE_NAME, DATE, VALUE
FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FREDDIE_MAC_HOUSING_TIMESERIES
WHERE VARIABLE_NAME = '30-Year Fixed Rate Mortgage Rate, National Average'
ORDER BY DATE DESC
LIMIT 12;
```

Part 2: Snowpark Connect (Prototype the Transform)

Snowpark Connect gives you a PySpark-like API that runs on Snowflake compute. Use it to prototype the transformation interactively • filter, combine, pivot • and see results before promoting to a Dynamic Table.

Step 2.0 - Create project database (run in Snowsight)

Create database, schema, and warehouse.

RUN IN: Snowsight SQL Worksheet

```
CREATE DATABASE IF NOT EXISTS ECON_AGENT_DB;
CREATE SCHEMA IF NOT EXISTS ECON_AGENT_DB.ANALYTICS;

CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;

USE DATABASE ECON_AGENT_DB;
USE SCHEMA ANALYTICS;
USE WAREHOUSE COMPUTE_WH;
```

Step 2.1 - Install Snowpark (local terminal)

RUN IN: local terminal

```
pip install "snowflake-snowpark-python[pandas]" snowflake-connector-python
```

Step 2.2 - Prototyping script (Python)

Run this in your local Python environment (or a notebook with Python kernel). Replace connection params with your account, user, and password. This prototypes the same logic we'll later put into a Dynamic Table.

RUN IN: local Python environment or notebook

```
from snowflake.snowpark import Session
from snowflake.snowpark import functions as F
from snowflake.snowpark.types import DoubleType

# -- 1. Connect to Snowflake -----
connection_params = {
    "account": "<your-account>", # e.g. "abc12345.us-east-1"
    "user": "<your-username>",
    "password": "<your-password>",
    "role": "ACCOUNTADMIN",
    "warehouse": "COMPUTE_WH",
    "database": "ECON_AGENT_DB",
    "schema": "ANALYTICS"
}

session = Session.builder.configs(connection_params).create()
print("Connected to Snowflake!")
```

```

# -- 2. Read the two source tables -----
econ = session.table(
  "SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES"
)
housing = session.table(
  "SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FREDDIE_MAC_HOUSING_TIMESERIES"
)

# -- 3. Extract CPI -----
cpi = (
  econ
  .filter(F.col("GEO_ID") == "country/USA")
  .filter(F.col("VARIABLE_NAME") == "CPI: All items, Monthly, 1982-84 Index Date (Seasonally adjusted)")
  .select(
    F.col("DATE"),
    F.col("VALUE").cast(DoubleType()).alias("VALUE"),
    F.lit("CPI").alias("METRIC")
  )
)
print("CPI rows:", cpi.count())

# -- 4. Extract Unemployment Rate -----
unemployment = (
  econ
  .filter(F.col("GEO_ID") == "country/USA")
  .filter(F.col("VARIABLE_NAME") == "Current Labor Force: Unemployment Rate - 20 yrs. & over, Monthly (Seasonally adjusted)")
  .select(
    F.col("DATE"),
    F.col("VALUE").cast(DoubleType()).alias("VALUE"),
    F.lit("UNEMPLOYMENT_RATE").alias("METRIC")
  )
)
print("Unemployment rows:", unemployment.count())

# -- 5. Extract Mortgage Rate (from a different table!) -----
mortgage = (
  housing
  .filter(F.col("VARIABLE_NAME") == "30-Year Fixed Rate Mortgage Rate, National Average")
  .select(
    F.col("DATE"),
    F.col("VALUE").cast(DoubleType()).alias("VALUE"),
    F.lit("MORTGAGE_RATE_30Y").alias("METRIC")
  )
)
print("Mortgage rows:", mortgage.count())

# -- 6. Union all three into one DataFrame -----
combined = cpi.union_all(unemployment).union_all(mortgage)
print("\nCombined data sample:")
combined.show(15)

# -- 7. Pivot: one row per date, one column per metric -----
dashboard = (
  combined
  .pivot("METRIC", ["CPI", "UNEMPLOYMENT_RATE", "MORTGAGE_RATE_30Y"])
  .agg(F.max("VALUE"))
  .sort(F.col("DATE").desc())
)

# -- 8. Preview the result -----
print("\nFinal pivoted dashboard:")
dashboard.show(20)
print(f"Total rows: {dashboard.count()}")
print(f"Columns: {dashboard.columns}")

session.close()
print("\nLooks good! Now we'll promote this to a Dynamic Table.")

```

Part 3: Dynamic Tables (Production Pipeline)

A Dynamic Table is like a materialized view that refreshes itself. You provide a SQL query and a target lag; Snowflake keeps it fresh automatically. No cron jobs or orchestrators.

Step 3.1 - Create the Dynamic Table (run in Snowsight)

Same logic as the Snowpark prototype • filter, union, pivot • written as SQL for the Dynamic Table.

The SQL query below creates or replaces a Dynamic Table in Snowflake called `ECONOMIC_DASHBOARD_LIVE`.

What this query does: • It compiles economic data for the US from multiple public sources into a single dashboard table. • Specifically, it pulls: • Consumer Price Index (CPI) data • Unemployment rate data • 30-Year Fixed Rate Mortgage Rate data • The query gathers these metrics from two public Snowflake tables: • `FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES` (CPI and Unemployment) • `FREDDIE_MAC_HOUSING_TIMESERIES` (Mortgage rates) • It transforms the different metrics into a common structure using `UNION ALL`, adds a label for each metric, and standardizes the value type. • The outer query pivots the combined data so that each `DATE` becomes a row, and each metric becomes a column. • The resulting Dynamic Table is always kept up-to-date by Snowflake (refreshing at least every hour), so analytic queries can be run on the most recent values without worrying about ETL, schedulers, or outdated results. • The three resulting columns-`CPI`, `UNEMPLOYMENT_RATE`, and `MORTGAGE_RATE_30Y`-show a concise snapshot of key US economic indicators for each date.

This setup lets you visualize and analyze these indicators in one place and is the foundation for building an AI agent or dashboard.

RUN IN: Snowsight SQL Worksheet

```
USE DATABASE ECON_AGENT_DB;
USE SCHEMA ANALYTICS;

CREATE OR REPLACE DYNAMIC TABLE ECONOMIC_DASHBOARD_LIVE
  TARGET_LAG = '1 hour'
  WAREHOUSE = COMPUTE_WH
AS
SELECT
  DATE,
  MAX(CASE WHEN METRIC = 'CPI' THEN VALUE END) AS CPI,
  MAX(CASE WHEN METRIC = 'UNEMPLOYMENT_RATE' THEN VALUE END) AS UNEMPLOYMENT_RATE,
  MAX(CASE WHEN METRIC = 'MORTGAGE_RATE_30Y' THEN VALUE END) AS MORTGAGE_RATE_30Y
FROM (
  -- CPI and Unemployment from the economic indicators table
  SELECT DATE, VALUE::DOUBLE AS VALUE, 'CPI' AS METRIC
  FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES
  WHERE GEO_ID = 'country/USA'
  AND VARIABLE_NAME = 'CPI: All items, Monthly, 1982-84 Index Date (Seasonally adjusted)'

  UNION ALL
  SELECT DATE, VALUE::DOUBLE, 'UNEMPLOYMENT_RATE'
  FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FINANCIAL_ECONOMIC_INDICATORS_TIMESERIES
  WHERE GEO_ID = 'country/USA'
  AND VARIABLE_NAME = 'Current Labor Force: Unemployment Rate - 20 yrs. & over, Monthly (Seasonally adjusted)'

  UNION ALL
  -- Mortgage rate from the Freddie Mac table
  SELECT DATE, VALUE::DOUBLE, 'MORTGAGE_RATE_30Y'
```

```
FROM SNOWFLAKE_PUBLIC_DATA_FREE.PUBLIC_DATA_FREE.FREDDIE_MAC_HOUSING_TIMESERIES
WHERE VARIABLE_NAME = '30-Year Fixed Rate Mortgage Rate, National Average'
)
GROUP BY DATE;
```

Step 3.2 - Verify (run in Snowsight)

Check the data and refresh status.

RUN IN: Snowsight SQL Worksheet

```
-- Check the data
SELECT * FROM ECONOMIC_DASHBOARD_LIVE
ORDER BY DATE DESC
LIMIT 20;

-- Check refresh status
SHOW DYNAMIC TABLES LIKE 'ECONOMIC_DASHBOARD_LIVE' IN SCHEMA ANALYTICS;
```

Part 4: Semantic Views (Teach AI Your Data)

A semantic view is a data dictionary in your schema • business-friendly names, descriptions, and metric definitions so Cortex Analyst knows how to interpret your data. Build it in the Snowsight wizard (AI & ML → Cortex Analyst → Create new Semantic View).

Step 4.1-4.4 - Create Semantic View in Snowsight

RUN IN: Snowflake Semantic View wizard

1. AI & ML → Cortex Analyst → Create new → Create new Semantic View
2. Location: ECON_AGENT_DB → ANALYTICS. Name: **ECONOMIC_DASHBOARD_SEMANTIC**
3. Description: *US economic indicators with 3 key metrics: CPI (inflation), unemployment rate, and 30-year mortgage rate. Sources: Bureau of Labor Statistics and Freddie Mac.*
4. Example questions: e.g. *What is the current unemployment rate?*, *How has CPI changed over the last 2 years?*, *What is the 30-year mortgage rate?*
5. Select table: **ECON_AGENT_DB.ANALYTICS.ECONOMIC_DASHBOARD_LIVE**
6. Select columns: DATE, CPI, UNEMPLOYMENT_RATE, MORTGAGE_RATE_30Y. Enable *Add sample values* and *Add AI-generated descriptions*.
7. Review and add details: CPI • *Consumer Price Index, base 1982-84=100. 324 ≈ 3.24×.* UNEMPLOYMENT_RATE • *Decimal; 0.041 = 4.1%. Multiply by 100 for %.* MORTGAGE_RATE_30Y • *Decimal; 0.0626 = 6.26%. Multiply by 100 for %.*
8. Create and save.

Step 4.5 - Verify (optional, run in Snowsight)

RUN IN: Snowsight SQL Worksheet

```
SHOW SEMANTIC VIEWS IN SCHEMA ECON_AGENT_DB.ANALYTICS;  
DESCRIBE SEMANTIC VIEW ECON_AGENT_DB.ANALYTICS.ECONOMIC_DASHBOARD_SEMANTIC;
```

Part 5: Cortex Analyst + Streamlit App

Everything is connected: Marketplace data → Dynamic Table → Semantic View → Cortex Analyst. You can use Cortex Analyst in Snowsight or wrap it in a Streamlit app.

Step 5A - Demo Cortex Analyst in Snowsight

RUN IN: Snowflake Cortex Analyst

AI & ML → Cortex Analyst → select `ECON_AGENT_DB.ANALYTICS.ECONOMIC_DASHBOARD_SEMANTIC` and ask: • *What is the most recent unemployment rate?* • *How has CPI changed over the last 2 years?* • *What is the 30-year mortgage rate now vs a year ago?*

Step 5B - Streamlit app (run in Snowflake Streamlit)

In Snowsight: Projects → Streamlit → + Streamlit App. Set app name *Ask the US Economy*, database/schema/warehouse (`ECON_AGENT_DB`, `ANALYTICS`, `COMPUTE_WH`). Add package `snowflake-ml-python`. Replace the default code with the following.

RUN IN: Snowflake Streamlit

```
import streamlit as st
from snowflake.snowpark.context import get_active_session
from snowflake.cortex import Complete

session = get_active_session()

st.title("■ Ask the US Economy")
st.caption("Powered by Snowflake Cortex & Dynamic Tables • Data: BLS & Freddie Mac")

# Pull latest data from our auto-refreshing Dynamic Table
@st.cache_data(ttl=600)
def load_data():
    return session.sql(
        "SELECT * FROM ECONOMIC_DASHBOARD_LIVE ORDER BY DATE DESC LIMIT 24"
    ).to_pandas()

data = load_data()

# Show a quick snapshot
col1, col2, col3 = st.columns(3)
latest = data.dropna(subset=["UNEMPLOYMENT_RATE"]).iloc[0] if not data.empty else None
if latest is not None:
    col1.metric("Unemployment", f"{latest['UNEMPLOYMENT_RATE']*100:.1f}%")
latest_cpi = data.dropna(subset=["CPI"]).iloc[0] if not data.empty else None
if latest_cpi is not None:
    col2.metric("CPI Index", f"{latest_cpi['CPI']:.1f}")
latest_mort = data.dropna(subset=["MORTGAGE_RATE_30Y"]).iloc[0] if not data.empty else None
if latest_mort is not None:
    col3.metric("30Y Mortgage", f"{latest_mort['MORTGAGE_RATE_30Y']*100:.2f}%")

st.divider()

# Chat interface
if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

if prompt := st.chat_input("Ask about unemployment, inflation, or mortgage rates..."):
```

```

st.session_state.messages.append({"role": "user", "content": prompt})
with st.chat_message("user"):
    st.markdown(prompt)

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            llm_prompt = f"""You are a US economic data analyst. Answer using ONLY this data.
            Be concise. Cite specific numbers and dates.

            DATA:
            {data.to_string(index=False)}

            NOTES:
            - CPI: Consumer Price Index (1982-84=100). Higher = more inflation.
            - UNEMPLOYMENT_RATE: Decimal. 0.041 = 4.1%.
            - MORTGAGE_RATE_30Y: Decimal. 0.0626 = 6.26%.

            QUESTION: {prompt}"""

            answer = Complete("mistral-large", llm_prompt)
            st.markdown(answer)
            st.session_state.messages.append({"role": "assistant", "content": answer})

with st.expander("■ View raw data"):
    st.dataframe(data, use_container_width=True)

```

Step 5C - Call Cortex Analyst from your own code (REST API)

Use this when you want to call the analyst from a Slack bot, mobile app, or backend. Install: `pip install snowflake-connector-python requests`

Replace **ACCOUNT**, **USER**, and **PASSWORD** with your credentials. Run in a local Python environment.

RUN IN: local Python environment

```

import requests
import snowflake.connector

ACCOUNT = "<your-account>" # e.g. "myorg-myaccount"
HOST = f"{ACCOUNT}.snowflakecomputing.com"
USER = "<your-username>"
PASSWORD = "<your-password>"
SEMANTIC_VIEW = "ECON_AGENT_DB.ANALYTICS.ECONOMIC_DASHBOARD_SEMANTIC"
WAREHOUSE = "COMPUTE_WH"
DATABASE = "ECON_AGENT_DB"
SCHEMA = "ANALYTICS"

# Step 1 - Get a session token for the REST API
login = requests.post(
    f"https://{HOST}/session/v1/login-request",
    json={"data": {"ACCOUNT_NAME": ACCOUNT, "LOGIN_NAME": USER, "PASSWORD": PASSWORD}},
    headers={"Content-Type": "application/json"},
)
token = login.json()["data"]["token"]

# Step 2 - Ask Cortex Analyst (REST API)
question = "What is the current unemployment rate?"
resp = requests.post(
    f"https://{HOST}/api/v2/cortex/analyst/message",
    json={
        "messages": [{"role": "user", "content": [{"type": "text", "text": question}]}],
        "semantic_view": SEMANTIC_VIEW,
    },
    headers={"Authorization": f'Snowflake Token="{token}"', "Content-Type": "application/json"},
)

```

```

)

# Step 3 - Parse the response
data = resp.json()
sql_statement = None
for block in data.get("message", {}).get("content", []):
    if block["type"] == "text":
        print("■ Analyst:", block["text"])
    elif block["type"] == "sql":
        sql_statement = block["statement"]
    print(f"\n■ Generated SQL:\n{sql_statement}")

# Step 4 - Execute the SQL with the Python connector
if sql_statement:
    conn = snowflake.connector.connect(
        account=ACCOUNT, user=USER, password=PASSWORD,
        warehouse=WAREHOUSE, database=DATABASE, schema=SCHEMA,
    )
    cur = conn.cursor()
    cur.execute(sql_statement)
    cols = [desc[0] for desc in cur.description]
    rows = cur.fetchall()
    print(f"\n■ Result:")
    print(" | ".join(cols))
    for row in rows:
        print(" | ".join(str(v) for v in row))
    cur.close()
    conn.close()

```

You're done

Same data pipeline. Two interfaces (Cortex Analyst in Snowsight + Streamlit app). Optional REST API for external apps. Zero external dependencies.

Completion

Your architecture at a glance

Marketplace data feeds a self-refreshing Dynamic Table. The Semantic View teaches Cortex Analyst how to interpret the metrics. You can then use either Cortex Analyst in Snowsight or the Streamlit interface, with the REST API available for external applications.

YOU ARE DONE

Same data pipeline. Two interfaces: Cortex Analyst in Snowsight and a Streamlit app. Optional REST API for external apps. Zero external dependencies.

Notebook source preserved and reorganized for practical use.